

PRODUCT MANAGEMENT SYSTEM

Complete Consolidated Design Document

Route Delivery & Exchange Operations Platform

A single reference document combining the full solution design, architecture blueprint, business value case, configuration engine, vehicle management, invoicing, and every product, business, compliance, and technical gap addressed to date.

Document Version: 1.0 (Consolidated) | Status: Design Complete, Pre-Development
Reference Seed Use Case: SMS Water Supply, North Chennai

Table of Contents

Part 1 — Solution Design: Core Insight, Domain Model & Roadmap

Part 2 — Complete Architecture Blueprint

Part 3 — Business Value, Multi-Vertical Workflow & Adoption Plan

Part 4 — Configuration Engine, Vehicle Management & Module List

Part 5 — Invoicing Sub-Module

Part 6 — Gap Resolution: Product, Business & Compliance Design

Part 7 — Technical Gap Pass: Reliability, Security & Quality Design

Part 8 — Status Summary & What's Next

This document is cumulative — each part was produced at a distinct stage of the design process and builds directly on the parts before it. Read sequentially for the full narrative, or jump to any part as a standalone reference.

PART 1

Solution Design

Core Insight, Domain Model, Workflow Engine & Roadmap

1.1 Executive Summary

Water-can delivery, LPG cylinder delivery, dairy subscriptions, and newspaper rounds are all instances of one underlying pattern: **recurring route-based delivery with unit exchange**. Rather than build a single-purpose water delivery app, this document defines a generalized **Product Management System** — a workflow engine plus a configurable data model — that can run any of these businesses from one codebase, differentiated purely by tenant-level configuration.

1.2 Core Insight: One Engine, Many Businesses

The difference between a water-can business and a dairy business is not logic — it is parameters. Both involve a party (customer), a route, a trip, a set of stops, product movement, and money movement. What changes is how the product behaves: exchanged, consumed, or owned by the customer long-term.

Business	Unit Type	Exchange Ratio	Typical Relationship
Water cans	exchange	1:1	Subscription / route
LPG cylinder	exchange	1:1	Subscription / route
Dairy (milk)	consumable	n/a	Subscription / route
Newspaper	consumable	n/a	Subscription / route
RO / filter service	deposit	n/a	Own-asset, service visits

One workflow engine, driven by these fields, replaces what would otherwise be five separate codebases — nothing about "cans" or "bottles" is hardcoded anywhere in the system.

1.3 Generalized Domain Model

Six core entities form the backbone of the system, all tenant-scoped:

- **Product** — unit_type: exchange, consumable, or deposit.
- **Party** — a customer, with relationship_type: subscription/route, ad-hoc/phone-in, or walk-in.
- **Inventory Ledger** — append-only log of every unit movement; the single source of truth for stock.
- **Route / Trip** — vehicle, staff, and ordered stops for a time window. Route belongs to the trip, never the staff member.
- **Stop Event** — atomic record of one delivery/collection: quantity given, quantity returned, payment collected.
- **Money Ledger** — the same append-only pattern applied to cash; every balance is a derived view, never a stored number.

Why this matters: the ledger-first approach is the single biggest professional upgrade over ad-hoc tracking. Any end-of-day mismatch is traceable to one row instead of becoming an unexplained discrepancy.

1.4 Workflow Engine

Trip Lifecycle

planned → loaded → dispatched → in_progress → completed → reconciled

Offline-First Stop Logging

Stop events are captured locally on the device and synced later. Each carries a client-generated unique ID so a retried sync can never double-count a delivery.

Automatic End-of-Day Reconciliation

- Expected stock = quantity loaded – quantity delivered + quantity returned
- Expected cash = sum of order values across all stops for the day
- Both are produced automatically from the ledgers; staff only confirm or flag variances.

Background Processing

Salary calculation, dues aging, and low-stock alerts run as background jobs, decoupled from the live request path.

1.5 Multi-Tenant Architecture

- **Tenant isolation** enforced at the data layer so no tenant can ever see another's data.
- **Tenant-level product catalog configuration** lets Tenant A run water and Tenant B run dairy on the exact same codebase.
- **Role model** generalizes across every vertical: Owner/Admin, Secondary Admin, Route Staff, and (future) Customer.

1.6 Phased Roadmap

Phase	Scope	Outcome
Phase 1	Core ops: parties, products, routes, trips, stop events, ledgers, EOD, salary basics	Fully operational single-tenant, single-vertical deployment
Phase 2	Customer portal, payment gateway, subscription billing, WhatsApp, distributor invoicing	Self-service customer experience layered on Phase 1
Phase 3	Multi-vertical activation, tenant self-signup with super-admin approval	True horizontal SaaS across recurring-delivery verticals

1.7 Practical Implementation Guidance

- **Schema first** — implement the six core entities before any business-specific screens.
- **Config over code** — every business-specific behavior is a data field, never an if/else branch.
- **Layer discipline** — strict separation between UI, workflow logic, and data access.
- **One step at a time** — build and verify in discrete increments, starting with Phase 1.

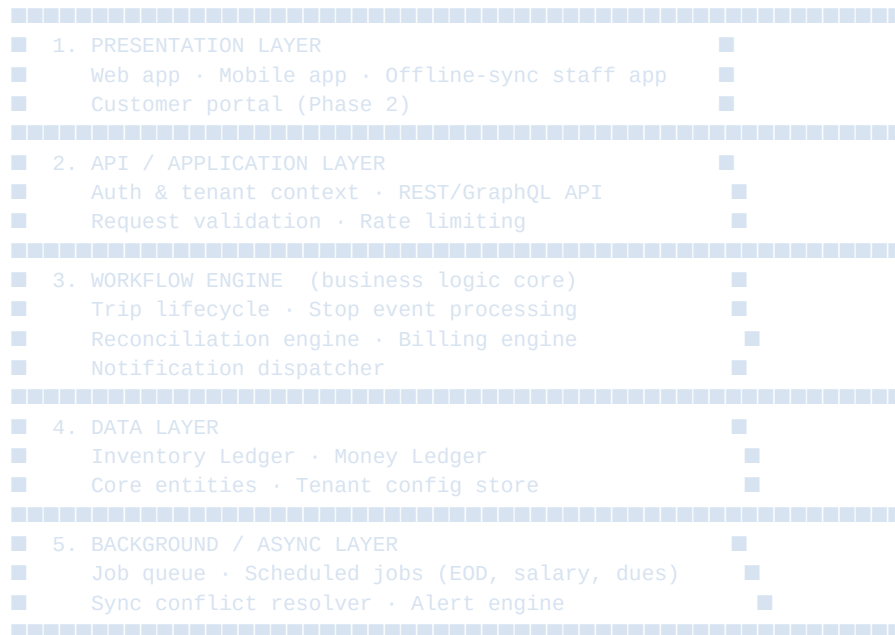
PART 2

Complete Architecture Blueprint

System Layers, Entity Model, API Design, Security & Deployment

2.1 System Architecture — Layered View

The system is organized into five layers, each depending only on the layer directly beneath it, allowing any single layer to change without cascading rewrites.



2.2 Complete Entity Model

Tenant & Identity

Entity	Key Fields	Purpose
Tenant	name, plan, active_status	One row per business using the platform
User	auth_identity, tenant_id, role	Login identity scoped to a tenant
TenantConfig	product_catalog_rules, exchange_ratios, feature_flags	Drives business-specific behavior via configuration, not code

Catalog

Entity	Key Fields	Purpose
Product	name, unit_type, case_size	Defines what is delivered and how it behaves
PriceList	product_id, tenant_id, effective_date, price	Time-versioned pricing, supports history and audits

Parties

Entity	Key Fields	Purpose
Party	name, address, geo, relationship_type	Any customer: route, ad-hoc, or walk-in
PartyProductSubscription	party_id, product_id, standing_qty, frequency	Standing order for route customers

Operations

Entity	Key Fields	Purpose
Vehicle	type, capacity, compliance_docs	Delivery vehicle fleet record
Staff	linked_user, role, vehicle_history	Delivery/route personnel
Trip	vehicle_id, staff_id, date, status	One vehicle's daily run
TripStop	trip_id, party_id, sequence, planned_qty	Planned stop within a trip
StopEvent	actual_qty_given/returned, amount_collected, client_uuid, gps_stamp	Ground-truth record of a delivery

Ledgers — Source of Truth (append-only)

Entity	Key Fields	Purpose
InventoryLedgerEntry	product_id, from_loc, to_loc, qty, event_type, timestamp	Every unit movement, ever
MoneyLedgerEntry	party_id/staff_id, amount, direction, reason, reference_id, timestamp	Every cash movement, ever

Derived Views (never stored as mutable state)

- CustomerBalanceView, StockOnHandView, StaffCashInHandView, DailyReconciliationView

Phase 2 — Schema-Ready, Dormant Until Activated

- Order, PaymentTransaction, Invoice, NotificationLog

2.3 API Design

Resource-oriented; every endpoint is implicitly tenant-scoped via the auth token, never a client-supplied parameter.

```
GET/POST /tenants/{id}/config
GET/POST /products
GET/POST /parties
GET/POST /vehicles, /staff, /trips
POST /stop-events (idempotent via client_uuid)
GET /ledger/inventory, /ledger/money (read-only)
GET /reports/eod, /reports/salary, /reports/dues-aging
```

2.4 Security Architecture

- Tenant isolation enforced at the data layer (row-level security)
- Phone OTP authentication for staff and owners
- Role-based access control across four generalized roles
- Audit trail by construction — ledger entries are immutable and append-only
- Offline sync integrity via client-generated UUIDs preventing duplicate writes

2.5 Deployment & Runtime Architecture

- Stateless application tier behind a load balancer, horizontally scalable
- Dedicated background job runner, decoupled from the live request path
- Offline-first mobile client with idempotent server-side merge
- Notification dispatcher (Phase 2) decoupled from core engine logic

2.6 Scalability & Multi-Vertical Activation

- New vertical = a new TenantConfig product catalog entry, zero code changes
- New tenant = a new isolated row set under the same schema and workflow engine
- Entity-generic reporting — never hardcoded to a business-specific concept

PART 3

Business Value, Multi-Vertical Workflow & Adoption

Why This Is a Genuine Change, and How Rollout Happens

3.1 Multi-Vertical Workflow Walkthrough

Business	unit_type	Exchange Ratio	Billing Pattern
Water Cans	exchange	1:1	Per-delivery cash / running dues
Dairy	consumable	n/a	Per-delivery or monthly
LPG Cylinder	exchange	1:1	Per-delivery cash / running dues
Newspaper	consumable	n/a	Monthly subscription billing

Every row above is produced by filling in a configuration form, not by writing new application code. A new vertical is a business decision, not an engineering project.

3.2 Why This Is a Genuinely Big Change

What People Do Today

- Paper notebooks, WhatsApp groups for orders, a calculator for month-end billing
- Or a generic warehouse ERP that has no concept of daily route delivery or exchange units

What Changes With This Platform

- **Same-day truth** — reconciliation runs automatically every evening
- **One system, many businesses** — same software, different configuration
- **New business type = a form, not a rewrite**
- **Every number is provable** — append-only ledgers trace any balance to its source rows
- **Accountability without confrontation** — expected-vs-actual shown automatically

Honest caveat: the real test is whether delivery staff reliably log stops in the field, and whether owners trust the numbers over years of notebook-built instinct. That adoption curve needs a deliberate plan, addressed below.

3.3 Adoption & Rollout Plan

Phase 0 — Single-Route Pilot (Weeks 1-2)

- Deploy to one delivery boy, one route only
- Run the paper notebook in parallel as a safety net

Phase 1 — Trust-Building Through Transparency (Weeks 3-4)

- Show the delivery boy his own reconciliation before the owner sees it
- Owner reviews mismatches together with staff, framed as collaborative, not accusatory

Phase 2 — Fleet-Wide Rollout (Weeks 5-8)

- Expand route-by-route, each shadowed by someone who already trusts the system
- Drop the paper notebook only after two consecutive clean weeks on that route

Phase 3 — Full Cutover & Second Vertical (Month 3+)

- Notebooks retired entirely for the seed tenant
- Onboard the next business type as the first proof point for configuration-not-code

Key Success Signals

Signal	What It Tells You
% of stops logged within the shift	Real-time adoption, not backfilled compliance
Average daily reconciliation mismatch	Should trend toward zero
Time from trip completed to reconciled	Should shrink week over week
Staff sentiment on logging effort	Extra work vs. felt protection

Configuration Engine, Vehicle Management & Modules

Billing Plans, Payroll, Fleet Management & the Complete Module Map

Configuration Engine: Billing, Payroll & Workflow Rules

Product behavior alone (`unit_type`, `exchange_ratio`) is not enough to support every business model. Monthly subscriptions, fixed monthly salaries, and commission-based pay all require billing and payroll to become independently configurable dimensions, separate from the delivery workflow itself. This section defines that layer.

1. Billing Cycle Engine

Money ledger entries are currently created per `StopEvent`, which only covers pay-per-delivery businesses. A new entity, **BillingPlan**, decouples billing from delivery:

- **billing_cycle** — `per_delivery` / `weekly` / `monthly` / `custom_days`
- **billing_trigger** — `on_delivery` / `on_calendar_date` / `on_usage_threshold`
- **amount_calculation** — `fixed_fee` / `per_unit_delivered` / `tiered`

A scheduled job reads active `BillingPlans`, calculates what is owed based on the trigger, and writes a `MoneyLedgerEntry` with `reason = subscription_invoice`. Delivery keeps logging `StopEvents` exactly as before — billing simply reads from those events differently depending on the plan. A newspaper business (monthly billing, daily delivery) and a water business (pay-per-can) run on the same ledger, differentiated only by `BillingPlan` configuration.

2. Payroll / Compensation Engine

The same pattern applies to staff pay via a new **CompensationPlan** entity:

- **pay_model** — `per_trip_commission` / `fixed_monthly` / `hybrid_base_plus_incentive`
- **pay_cycle** — `daily` / `weekly` / `monthly`
- **calculation_basis** — `trips_completed` / `units_delivered` / `fixed_amount` / `attendance_days`

A `PayrollRun` background job reads each staff member's `CompensationPlan`, pulls the relevant numbers from the `Inventory/Money` ledgers, and writes salary disbursement as a `MoneyLedgerEntry` with `reason = salary_disbursement`. One staff member can be on fixed monthly salary while another is on per-trip commission — same engine, different configuration row.

3. The Tenant Rules Layer

This is the complete answer to configuring all delivery systems and ERP logic in one place: a dedicated configuration layer sitting above the core entities, filled in once per tenant during onboarding.

Configuration Domain	What the Owner Sets
Product behavior	<code>unit_type</code> , <code>exchange_ratio</code> , <code>case_size</code>
Customer billing	<code>billing_cycle</code> , <code>trigger</code> , <code>calculation method</code>
Staff compensation	<code>pay_model</code> , <code>pay_cycle</code> , <code>calculation_basis</code>
Workflow triggers	<code>subscription renewal dates</code> , <code>invoice timing</code> , <code>low-stock thresholds</code>
Notification rules	<code>when to alert customer/staff/owner</code> , and <code>via which channel</code>

The Workflow Engine, Billing Engine, and Payroll Engine are all interpreters that read this configuration and act. They contain zero business-specific logic — every difference between a water business and a gym membership business is a data row, not a code path.

4. Onboarding Presets (avoiding configuration overload)

Full configurability creates a UX risk: a new business owner will not want to fill in ten configuration screens on day one. The mitigation is **vertical presets** that pre-fill sensible defaults, which the owner can then fine-tune:

- "Water / Gas Delivery" preset — pre-fills exchange billing + per-trip commission
- "Subscription Service" preset — pre-fills monthly billing + fixed salary
- "Mixed / Custom" — opens the full configuration form for advanced tenants

Vehicle Management Module

Vehicles are a first-class operational asset, not just a foreign key on a Trip. This module covers the full vehicle lifecycle from onboarding to cost analysis.

Core Entities

Entity	Key Fields	Purpose
Vehicle	type, capacity, registration_no, ownership_type	Master record of every fleet asset
VehicleComplianceDoc	doc_type (RC/insurance/permit/PUC/FSSAI), expiry_date, file_ref	Tracks legal documents with expiry alerting
VehicleMaintenanceLog	service_date, odometer, cost, workshop, notes	Service history and upcoming maintenance due
VehicleFuelLog	date, litres, cost, odometer	Fuel expense and mileage tracking per vehicle
VehicleAssignmentHistory	vehicle_id, staff_id, trip_id, date	Who drove which vehicle when — derived from Trips, not a fixed assignment
VehicleStatus	availability (active/in_service/breakdown/retired)	Real-time fleet availability for trip planning

Key Workflows

- **Compliance alerting** — background job checks VehicleComplianceDoc expiry dates and notifies the owner before a vehicle becomes legally non-compliant to operate
- **Trip eligibility check** — a vehicle with expired compliance or 'breakdown' status is automatically excluded from trip planning, preventing operational and legal risk
- **Cost-per-vehicle reporting** — fuel + maintenance costs rolled up per vehicle, cross-referenced against trips/units delivered, to surface which vehicles are profitable
- **Driver-vehicle flexibility** — since assignment is derived from Trip records rather than fixed, the same staff member can drive different vehicles on different days without a schema change

Complete Module List for End-to-End Workflow

Bringing everything together, a complete platform for recurring-delivery businesses needs the following modules. Modules already covered in depth in prior sections are marked accordingly; new modules required for full ERP-level coverage are called out explicitly.

Foundation

Module	Status	Covers
Tenant & User Management	Covered	Multi-tenancy, roles, authentication, RLS isolation
Product & Catalog Management	Covered	Products, unit types, pricing history
Party / Customer Management	Covered	Customers, relationship types, subscriptions
Configuration / Rules Engine	Covered (this doc)	Billing plans, compensation plans, workflow triggers

Operations

Module	Status	Covers
Vehicle & Fleet Management	Covered (this doc)	Vehicle master, compliance, maintenance, fuel, cost analysis
Route & Trip Management	Covered	Trip lifecycle, stop planning, sequencing
Delivery / Stop Execution	Covered	Offline-first stop logging, idempotent sync
Inventory & Stock Management	New — needed	Godown stock, inbound from supplier/company, stock transfer between locations, wastage/damage tracking
Procurement / Purchase Management	New — needed	Purchase orders to suppliers, lorry receipt records, supplier payment tracking

Finance

Module	Status	Covers
Billing & Subscription Engine	Covered (this doc)	Per-delivery, weekly, monthly billing cycles
Payroll / Compensation Engine	Covered (this doc)	Fixed salary, commission, hybrid pay models
Accounts & Financial Ledger	New — needed	Full expense tracking (fuel, maintenance, rent), profit & loss reporting, tax-ready exports
Dues & Collections Management	Partially covered	Aging reports exist; needs dedicated follow-up/reminder workflow for overdue accounts

Engagement & Compliance

Module	Status	Covers
Notification & Communication	Covered (Phase 2)	WhatsApp/SMS dispatch, decoupled provider integration
Customer Portal / Self-Service	Covered (Phase 2)	Order placement, balance view, payment

Module	Status	Covers
Complaint / Service Ticket Management	New — needed	Breakage reports, quality complaints, service requests, resolution tracking
Document & Compliance Management	New — needed	Business licenses (e.g. FSSAI for water), staff ID proofs, contracts — beyond just vehicle docs
Reporting & Analytics Dashboard	Covered	EOD reconciliation, dues aging, salary, business-level dashboards
Audit & Security Logging	Covered	Immutable ledger trail, access logs

Net new modules to plan for: *Inventory & Stock Management (godown-level, not just in-trip), Procurement/Purchase Management, full Accounts/P&L, a dedicated Collections follow-up workflow, Complaint/Service Tickets, and broader Document/Compliance Management beyond vehicles. These complete the loop from supplier purchase through to customer collection and legal compliance — the full end-to-end ERP scope.*

PART 5

Invoicing Sub-Module

Generation Flow, Fields, Layout & Delivery Channels

Invoicing Sub-Module (Billing Engine)

Invoicing is not a separate system — it is the natural output of the Billing Cycle Engine already defined. The Invoice entity was flagged as schema-ready in the original data model; this section formalizes how it is generated, structured, and delivered.

1. Generation Flow

- BillingPlan job runs on its configured cycle (per_delivery / weekly / monthly) and calculates the amount owed for the period
- Instead of only writing a MoneyLedgerEntry, it also generates an Invoice record referencing the specific ledger entries that make it up — so every invoice line is traceable back to a real StopEvent or ledger row, never a manually typed figure
- The Invoice is rendered as a PDF using an open-source toolchain and made available to the customer through the configured delivery channel

2. Invoice Entity — Fields

Field	Purpose
invoice_number	Sequential, tenant-scoped identifier (e.g. SMS/2026/000482)
party_id	Which customer the invoice belongs to
billing_period	Date range for monthly/weekly plans; single date for per-delivery
line_items	Pulled from StopEvents/MoneyLedgerEntries — product, qty, rate, amount
tax_details	Optional GST breakdown — only for tenants registered for tax
total_amount / amount_paid / balance_due	Derived from the money ledger, never a stored editable number
status	draft / issued / paid / overdue

3. Sample Invoice Layout

[Tenant Business Name]			
[Address / Phone]		INVOICE SMS/2026/000482	
		Period: 01 Aug - 31 Aug	
Bill To: [Customer Name]			
[Customer Address]			
Product	Qty	Rate	Amount
20L Water Can	62	40	2,480
Subtotal			2,480
Tax (if applicable)			0
TOTAL DUE			2,480
Previous Balance			120
Amount Paid This Period			2,200
BALANCE CARRIED FORWARD			400
Status: ISSUED Generated automatically from			
delivery records – every line traceable to a stop.			

4. Delivery Channels

- **WhatsApp** — auto-sent as a PDF via the notification dispatcher (Phase 2)
- **Print** — for godown walk-in customers wanting a paper copy
- **Customer portal download** (Phase 2) — self-service access to full invoice history

5. Configuration Decisions Per Tenant

- **Tax applicability** — GST fields are optional per tenant; most small local delivery businesses are not GST-registered, so this defaults off and is enabled only when needed
- **Invoice vs. receipt** — a formal tax invoice has stricter numbering and format rules than a plain payment receipt; tenants default to "receipt" mode and can upgrade to "invoice" mode if they are a registered business

Because invoices are generated entirely from ledger data rather than manual entry, every figure on a customer's invoice is provable — the same principle that makes end-of-day reconciliation trustworthy now extends to customer-facing billing.

PART 6

Gap Resolution

Product, Business & Full India Compliance Design

Gap Resolution: Product, Business & Compliance Design

This section closes the product and business gaps identified in the platform review, and provides a full detailed design for India-specific compliance requirements (GST e-invoicing, DPDP Act 2023, TDS). Remaining technical gaps (backup/DR, offline conflict resolution, API rate limiting, monitoring, Tally export, QA strategy) are listed at the end as the next pass.

1. Platform Monetization Model

A new entity, **PlatformSubscription**, governs how the platform itself is billed to tenants — separate from **BillingPlan**, which governs how tenants bill their own customers.

Field	Purpose
plan_tier	free / growth / pro / enterprise
billing_cycle	monthly / annual
status	trial / active / past_due / cancelled
trial_end_date	Free trial conversion tracking

Suggested Tier Structure

- **Free** — 1 vehicle, 1 route, up to ~50 customers — sufficient for the pilot phase
- **Growth** — multiple vehicles/routes, up to ~500 customers, WhatsApp notifications included
- **Pro** — unlimited scale, multi-vertical (e.g. dairy + water on one tenant), priority support
- **Enterprise** — multi-branch operations, custom SLAs

Recommended billing basis: start with a flat fee per tier for simplicity; usage-based (per active customer) is fairer at scale but adds billing complexity best deferred. Payment collection from tenants can use Razorpay or Cashfree — both India-first and pay-as-you-go, fitting the free-tier-first deployment strategy already chosen.

2. Onboarding Wizard

A guided setup flow makes the configuration layer usable by a real small-business owner in one sitting, rather than presenting a blank configuration form.

- Step 1 — Business type selection, loads a preset (Water/Gas, Subscription Service, Custom)
- Step 2 — Business info: name, address, GST number (optional)
- Step 3 — Add first vehicle(s)
- Step 4 — Add first staff/route boy
- Step 5 — Product catalog, pre-filled from preset, editable
- Step 6 — Add first customers, manual entry or import
- Step 7 — Billing + compensation plan, preset defaults shown, editable
- Step 8 — Dashboard tour, onboarding complete

3. Data Import Tool

- CSV/Excel templates for customers, staff, vehicles, and standing orders
- Column-mapping step so an owner's existing spreadsheet does not need to match exact field names

- Validation preview showing errors before committing the import, so nothing silently corrupts the ledger on day one
- Built on open-source parsing (Python openpyxl/pandas on the backend) — no paid import service needed

4. Multi-Language Support

- i18n via react-i18next (open source) across both the staff mobile app and owner dashboard
- Priority order: Tamil first (seed tenant, North Chennai staff), English default, extensible to Hindi and other regional languages as tenants expand
- Staff-facing screens get translation priority over owner/admin screens
- Notification templates (WhatsApp messages, invoices) support per-tenant language selection

5. Customer Support / Helpdesk (Platform Level)

- Lightweight open-source helpdesk (Chatwoot) rather than a paid support tool
- WhatsApp support channel for tenant owners, matching the channel they already use with their own customers
- Three-tier escalation: L1 self-serve FAQ/knowledge base, L2 chat support, L3 developer escalation
- Informal SLA during pilot phase (e.g. respond within 24 hours) to set expectations early

6. Training Materials

- Short video walkthroughs in Tamil and English for the staff app
- Laminated one-page quick-reference card for delivery staff, usable without a phone reference
- Written EOD reconciliation review guide for owners
- Train-the-trainer approach: one power-user per pilot route trained deeply, becomes local support

India Compliance — Full Detailed Design

1. GST E-Invoicing

Field (GSTInvoiceMeta)	Purpose
invoice_id	Links to the parent Invoice record
irn	Invoice Reference Number issued by the GST e-invoice system
ack_no / ack_date	Acknowledgement reference and date
qr_code	QR code appended to the invoice PDF
einvoice_status	pending / generated / failed

Applicability is restricted to tenants who are GST-registered and cross the e-invoicing turnover threshold — this stays completely dormant for most small water/dairy tenants below threshold. Integration runs through a GST Suvidha Provider (GSP) API, which is inherently a paid third-party service, so it is built as a pluggable module activated only per-tenant when needed. Workflow: Invoice generated -> if tenant is GST-registered and above threshold -> call GSP API -> store IRN and QR code -> append to the invoice PDF.

2. DPDP Act 2023 (India's Data Protection Law)

- **Consent capture** — a checkbox at customer creation, logged in a new ConsentLog entity, tied to the specific data being collected (phone, address, payment history)
- **Purpose limitation** — each collected field is documented against why it is needed (address for delivery, phone for OTP/notifications)
- **Data minimization** — no field is added to Party unless it serves an operational purpose
- **Right to erasure** — since ledgers are append-only and cannot be deleted for accounting/legal retention, erasure means anonymizing PII fields on the Party record (name, phone, address redacted) while keeping financial ledger rows intact under an anonymized reference
- **Data localization** — store on India-region infrastructure; relevant both for the current free-tier cloud choice and the later self-hosted VPS location
- **Breach notification runbook** — a lightweight internal document (who to notify, what timeline) satisfies early-stage compliance readiness
- **Grievance Officer contact** — required at larger scale; treated as a "when you grow" requirement rather than a day-one blocker

3. TDS on Salary Disbursement

- New fields: CompensationPlan.tds_applicable (boolean), Staff.pan_number
- PayrollRun computes TDS deduction when applicable, recorded as its own MoneyLedgerEntry (reason = tds_deduction) — gross vs. net pay stays fully traceable, never a single opaque number
- Generates Form 16/16A-ready structured data — not a certified filing tool, just clean exportable numbers for an accountant
- For a seed tenant with likely informal staff below threshold, this stays optional and dormant

4. Deferred Technical Gaps (Next Pass)

The following remain open and should be addressed immediately after this product/compliance pass, before Sprint 0 begins, since they influence schema and infrastructure choices:

- Backup and disaster recovery strategy
- Offline sync conflict resolution beyond basic idempotency (concurrent edits, late-arriving stops after reconciliation)
- API rate limiting and abuse protection
- Monitoring and alerting thresholds (e.g. failed EOD reconciliation jobs)
- Tally export for tenant accountants
- Formal QA/testing strategy, especially unit tests on ledger correctness

With this pass complete, the platform design covers product usability, monetization, and India-specific legal compliance end to end. The remaining technical gap list is the recommended next step before development begins.

PART 7

Technical Gap Pass

Reliability, Security & Quality Design

Technical Gap Pass: Reliability, Security & Quality Design

This section closes the six technical gaps identified after the product/business and compliance pass: backup and disaster recovery, offline sync conflict resolution, API rate limiting, monitoring/alerting, Tally export, and QA/testing strategy. Closing these before Sprint 0 avoids rework, since each one influences schema or infrastructure decisions.

1. Backup & Disaster Recovery Strategy

Target	Value
RPO (Recovery Point Objective)	15 minutes — near-continuous protection given financial data
RTO (Recovery Time Objective)	4 hours for full platform restore

- Daily full backups plus continuous WAL archiving via pgBackRest or WAL-G (both open source)
- Backups stored encrypted, offsite, on a different provider than the primary database (e.g. Backblaze B2 — S3-compatible, open tooling)
- Retention: daily backups for 30 days, weekly for 3 months, monthly for 1 year, reflecting the legal/accounting retention implications of ledger data
- Quarterly restore drills into a staging environment, verifying ledger integrity post-restore — an untested backup is not considered a valid backup
- A logical per-tenant export/import tool is needed alongside full-DB restore, since a shared-DB-with-RLS restore is DB-wide by default and one tenant's issue should not require restoring every tenant

Free-tier note: Supabase's free tier does not guarantee point-in-time recovery. Given backups are non-negotiable, this is a candidate area to budget a small paid tier for even during the free-tier deployment phase, or to self-manage backups from day one.

2. Offline Sync Conflict Resolution

Scenario	Resolution
Same stop event synced twice with edited data	Server timestamp determines the accepted version; the correction is appended as a new adjustment entry referencing the original — never an overwrite
Stop logged after the trip is already reconciled	Reopened as a flagged exception, never silently folded into the closed EOD report; creates a dated post-reconciliation adjustment entry
Two devices log the same trip stop concurrently (e.g. phone handed to a reliever mid-shift)	First successful write (by client_uuid) wins; the second is rejected with 'already logged' and the client shows the existing record instead of erroring silently

Sync protocol is queue-based with exponential backoff retry; the server validates trip status before accepting normal writes (must be in_progress or completed, not reconciled). Because the ledger is append-only, every correction is a new row, never an UPDATE — this is what keeps the audit trail intact even when field conditions get messy.

3. API Rate Limiting & Abuse Protection

- Rate limits enforced per-tenant and per-user at the gateway (e.g. 100 req/min per staff device, higher for admin dashboards)

- Open-source implementation via Traefik or Kong as API gateway, or lightweight middleware (express-rate-limit / slowapi) backed by Redis
- Failed-OTP abuse detection with exponential backoff lockout
- Tenant-level quotas tied to PlatformSubscription tier — free tier gets a lower API ceiling, directly connecting rate limiting to the monetization model
- Cloudflare free tier in front of the app for basic DDoS protection, unchanged after migrating to a self-hosted VPS

4. Monitoring & Alerting Thresholds

Core metrics tracked: API error rate, API latency (p95/p99), DB connection pool saturation, background job failure rate, and EOD reconciliation completion status per tenant per day.

Condition	Alert Action
EOD reconciliation job fails or misses cutoff (e.g. 11 PM)	Page on-call
API error rate exceeds 5% over a 5-minute window	Alert
DB storage exceeds 80% capacity	Alert
Offline app sync failure rate exceeds 10%	Alert — signals connectivity or bug issue

Tooling: Grafana + Prometheus (already chosen for the stack), Alertmanager for routing, notifications via free-tier email or a Telegram bot — avoiding a paid PagerDuty subscription at this stage of the platform.

5. Tally Export

- Most small Indian businesses' accountants work in Tally or Tally-compatible tools like Vyapar
- A new ReportExport capability generates Tally-compatible XML from MoneyLedgerEntry and Invoice records, mapped to Tally voucher types (Sales, Receipt, Payment, Journal)
- CSV export offered as a fallback for accountants on other tools (Vyapar, Zoho Books, Busy)
- Deliberately export-only (one-way), avoiding the much larger scope of a live two-way accounting sync

6. QA / Testing Strategy

Ledger correctness receives the highest testing priority — this is where a bug directly costs someone money.

- Unit tests on every ledger-writing function (stop event processing, billing run, payroll run), covering edge cases: partial returns, zero quantity, negative adjustments
- Property-based testing (e.g. Hypothesis in Python) verifying the core invariant holds across random valid inputs: loaded = delivered + returned + remaining, always
- Integration tests covering the full trip lifecycle end-to-end (planned through reconciled) against a real test database
- Regression suite blocking merge/deploy if any ledger-critical test fails
- UAT — the Phase 0 pilot route doubles as real-world UAT, formalized with a checklist of workflows pilot staff must complete successfully before Phase 1 begins
- Load testing via k6 (open source), simulating expected concurrent trip volume before scaling past the pilot tenant

With this pass complete, all identified product, business, compliance, and technical gaps are closed. The design is now ready for Sprint 0: locking the DDL for Tenant through Money Ledger as the first concrete development step.

PART 8

Status Summary & What's Next

Design Complete — Ready for Sprint 0

Current Status

All identified product, business, compliance, and technical gaps have been addressed. The platform design — solution design, architecture blueprint, configuration engine, vehicle management, invoicing, and every gap pass — is complete and consistent.

Immediate Next Step: Sprint 0

Lock the DDL for the foundational schema (Tenant through Money Ledger) as actual database definitions, since every other module reads from this schema. This is the first concrete development step, to be done before any UI work begins.

Full Development Sequence

Sprint	Scope
Sprint 0 (Wks 1-2)	Tenant, User, Auth, RLS, Product, Party — schema foundation
Sprint 1 (Wks 3-5)	Vehicle, Route, Trip, StopEvent, Inventory + Money Ledger, offline logging
Sprint 2 (Wks 6-7)	EOD reconciliation job, derived views — first trustworthy output
Sprint 3 (Wks 8-10)	BillingPlan, Invoice, CompensationPlan, Payroll, notifications
Sprint 4 (Wks 11-12)	Vehicle compliance/maintenance, dashboards, reporting
Sprint 5 (Wks 13-14)	Backup, monitoring, load testing — then begin Phase 0 user pilot

This consolidated document represents the complete pre-development design for the Product Management System as of this stage. Any future design decisions should be appended as a new part, keeping this document the single source of truth for the platform's design.